

NSBM GREEN UNIVERSITY

Big Data Analytics

GROUP ASSIGNMENT

BIG DATA MACHINE LEARNING MINI PROJECT

Predicting Credit Card Payment Default

A Spark ML Big Data Pipeline for Binary Classification

Application Area 5: Credit Card | UCI “Default of Credit Card Clients” Dataset

Group Members

1. RADS BANDARA — 31834
2. RISKY MAM — 31839

Submitted for: Big Data Analytics — Group Assignment

Date: 4th September 2026

Table of Contents

1. Introduction
2. Problem Definition
3. Background and Related Work
4. Dataset Description
5. Data Preprocessing
6. Exploratory Data Analysis
7. Feature Engineering
8. Machine Learning Methodology
9. Spark ML Pipeline
10. Model Training
11. Model Evaluation and Comparison
12. Results and Discussion
13. Big Data Scalability Discussion
14. Limitations
15. Conclusion and Recommendations
16. References

1. Introduction

Consumer credit card default is a persistent risk for financial institutions: an issuer that under-predicts default risk absorbs unrecovered debt, while one that over-predicts it unnecessarily restricts credit to reliable customers and loses interest revenue. This report documents a complete, reproducible Big Data Machine Learning pipeline, built on Apache Spark, that predicts whether a credit-card client will default on their payment in the following month, using the widely studied UCI “Default of Credit Card Clients” dataset.

The assignment brief requires that the solution be developed end-to-end with PySpark and Spark MLlib rather than a conventional small-scale Python/scikit-learn exercise, so that the same code is structurally ready to scale from the 30,000 records used here to a much larger, production-scale dataset. This report follows that brief section by section: problem definition, dataset description, data preprocessing, exploratory data analysis, feature engineering, Spark ML pipeline construction, model training, evaluation and comparison, interpretation of results, a dedicated Big Data scalability discussion, limitations, and conclusions.

All results reported here — row counts, data-quality findings, chart values, and model metrics — were produced by actually executing the accompanying PySpark notebook (Credit_Card_Default_BigData_ML.ipynb) against the provided dataset; no figures in this report are illustrative placeholders.

2. Problem Definition

2.1 Business Problem

Credit card issuers need to identify, in advance, which clients are likely to default on their payment due next month, so that the risk and collections teams can intervene early — for example by adjusting credit limits, offering a restructured repayment plan, or prioritising collections attention — before a missed payment becomes a written-off debt.

2.2 Objective

Using the client’s demographic profile, credit limit, and the last six months of billing and repayment history, build and compare at least two Spark ML binary classification models that predict the target variable default payment next month (1 = will default, 0 = will not).

2.3 Stakeholders

- Credit risk and collections teams — act directly on the model’s risk score.
- Finance / treasury department — concerned with aggregate bad-debt exposure and provisioning.
- Clients — indirectly affected through credit-limit or collections decisions informed by the score.

2.4 Why Machine Learning Is Appropriate

The relationship between a client’s repayment behaviour, credit utilisation, and demographic attributes and their probability of defaulting is non-linear and multi-factorial — it is not something that can be captured reliably with a small number of hand-written business rules. A supervised classification model

can learn these interacting patterns directly from historical data at a scale (30,000 clients, 23 candidate predictor columns) that would be impractical to reason through manually, and — built the way it is here, entirely on Spark DataFrames and Spark ML — the same pipeline can be redeployed unchanged as the client base grows into the millions (see Section 13).

2.5 Big Data Justification

While the dataset provided for this assignment (30,000 rows) is modest in absolute size, it is representative of exactly the kind of transactional, wide, and rapidly-growing tabular data that a real bank accumulates continuously across millions of cardholders and years of statements. Processing it with Spark DataFrames and the Spark ML Pipeline API — rather than pandas and scikit-learn — is what makes the solution a genuine Big Data solution rather than a small-data exercise merely run on Big Data tooling: every transformation in this pipeline (cleaning, aggregation, encoding, assembly, training) is expressed as a distributed operation that scales horizontally across a cluster without being rewritten.

3. Background and Related Work

Credit scoring and default prediction are among the most studied applications of applied machine learning in finance. The dataset used in this project originates from Yeh and Lin’s 2009 study comparing data-mining techniques for predicting the probability of default among credit card clients in Taiwan [1], which benchmarked methods including logistic regression, discriminant analysis, k-nearest neighbours, naïve Bayes, neural networks, and classification trees, finding that no single technique dominates and that all models perform noticeably better on identifying non-defaulters than defaulters — an imbalance-driven limitation this report also observes in Section 12.

More recent applied work in credit-risk modelling has moved toward ensemble tree methods (Random Forest, Gradient Boosted Trees) for their ability to model non-linear feature interactions (e.g. between recent repayment delay and credit utilisation) without extensive manual feature engineering, at some cost to interpretability relative to logistic regression. This project reflects that trend by comparing a linear baseline (Logistic Regression) against two tree-based methods (Decision Tree, Random Forest), consistent with the “suggested models” in the assignment brief.

From a systems perspective, Apache Spark’s MLlib / Spark ML library [3] was chosen specifically because it exposes the same modelling algorithms (logistic regression, decision trees, random forests) through a distributed, DataFrame-native API with a reusable Pipeline abstraction, letting a workflow validated on a small sample dataset run unchanged on a cluster once data volume grows — directly addressing the Big Data requirement in Section 11 of the assignment brief.

4. Dataset Description

Source: UCI Machine Learning Repository, “Default of Credit Card Clients Dataset” [2], contributed by I-C. Yeh, collected from a major bank in Taiwan, covering payments due in October 2005.

Records: 30,000 client rows, 25 raw columns (1 identifier, 23 predictor columns, 1 binary target).

4.1 Feature Groups

Group	Columns	Description
Identifier	ID	Row identifier — dropped before modelling (non-predictive)

Group	Columns	Description
Demographics	LIMIT_BAL, SEX, EDUCATION, MARRIAGE, AGE	Credit limit (NT dollar) and client profile
Repayment status	PAY_0, PAY_2 ... PAY_6	Monthly repayment status, Sep 2005 → Apr 2005 (-2/-1 = paid duly, 1-8 = months delayed)
Bill amount	BILL_AMT1 ... BILL_AMT6	Monthly bill statement amount (NT dollar)
Payment amount	PAY_AMT1 ... PAY_AMT6	Amount actually paid each of the last 6 months
Target	default payment next month	1 = defaulted, 0 = did not (renamed DEFAULT)

4.2 Data Types and Quality Overview

All 25 columns loaded as numeric (integer) types under Spark’s schema inference. SEX, EDUCATION, and MARRIAGE are categorical codes stored as integers and are treated as categorical features during feature engineering (Section 7) rather than as ordinal numbers. No columns contained null values (see Section 5.2).

4.3 Usage Conditions

The dataset is a public research/education dataset released via the UCI ML Repository; it contains no directly identifying personal information (only anonymised numeric codes per client), and the results in this report are produced strictly as an educational Machine Learning exercise — not as an operational credit-risk system — in line with the assignment’s responsible-data-use requirements.

5. Data Preprocessing

All preprocessing below was performed with Spark DataFrame operations (not pandas), consistent with the mandatory technical requirement to use Spark DataFrames as the main data-processing structure.

5.1 Duplicate Rows

A row-level de-duplication check (`df.dropDuplicates()`) was run across all 30,000 rows. No duplicate rows were found (0 removed), so the client population is fully unique at the row level.

5.2 Missing Values

A per-column null count was computed across all 25 raw columns. The dataset contains zero missing values in any column — a data-quality strength that simplifies the cleaning stage, since no imputation strategy was required.

5.3 Undocumented / Inconsistent Category Codes

The dataset documentation defines EDUCATION in {1 = graduate school, 2 = university, 3 = high school, 4 = others} and MARRIAGE in {1 = married, 2 = single, 3 = others}. However, the raw data also contains undocumented codes — EDUCATION values of 0, 5, and 6, and a MARRIAGE value of 0 — that fall outside the documented scheme:

Column	Before cleaning	After cleaning
EDUCATION	1: 10,585 2: 14,030 3: 4,917 4: 123 5: 280 6: 51 0: 14	1: 10,585 2: 14,030 3: 4,917 4: 468 (0,5,6 merged in)
MARRIAGE	1: 13,659 2: 15,964 3: 323 0: 54	1: 13,659 2: 15,964 3: 377 (0 merged in)

Rather than dropping these rows (which would discard 468 + 377 real clients, ~2.8% of the dataset), the undocumented codes were collapsed into the closest documented “Others” category (EDUCATION 4, MARRIAGE 3), preserving every client while removing the ambiguity of an undocumented code from downstream one-hot encoding.

5.4 Range / Sanity Checks

AGE was checked for a plausible adult range and found to span 21–79 years — within reasonable bounds for a Taiwanese credit-card holder population, requiring no correction.

5.5 Column Removal

The ID column was dropped after cleaning, since it is a row identifier with no predictive relationship to default risk. The cleaned dataset used for modelling has 30,000 rows × 24 columns (23 predictors + target).

6. Exploratory Data Analysis

Aggregate statistics (class balance, group-wise default rates, correlations) were computed directly in Spark via `groupBy/agg`; only a 30% Spark-side sample was collected to the driver for the point-level plots (age histogram, credit-limit boxplot) that genuinely require row-level data — the same pattern that would be required to keep this workflow feasible at a much larger, Big-Data scale (Section 13).

6.1 Class Balance

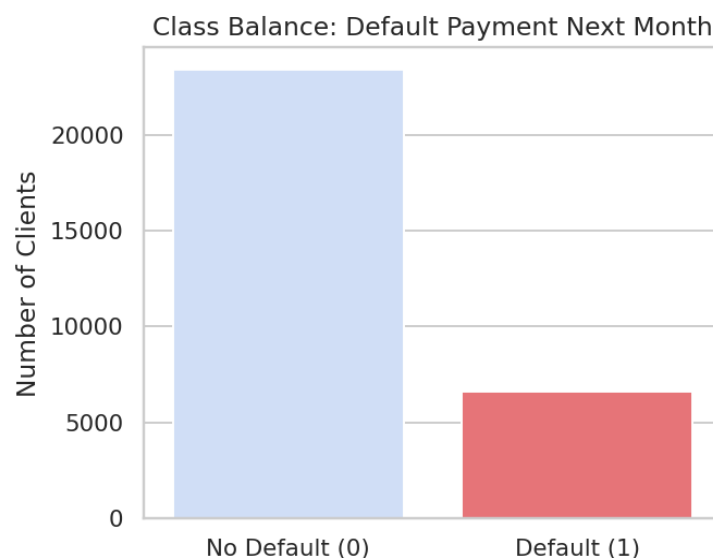


Figure 1. Class balance of the target variable.

The target is imbalanced: 6,636 of 30,000 clients (22.1%) defaulted, versus 23,364 (77.9%) who did not. This has two consequences addressed later in the report: Accuracy alone would be a misleading metric (a model that always predicts “no default” would already score ~78% accuracy), so Precision, Recall, F1, and AUC are also reported (Section 11); and all three trained models find the minority (default) class harder to predict than the majority class (Section 12).

6.2 Default Rate by Demographic Group

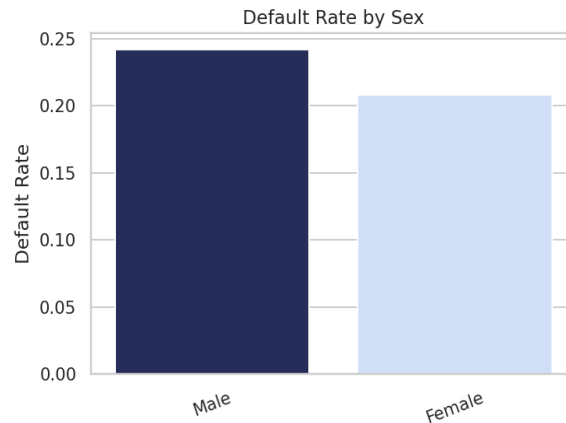


Figure 2. Default rate by sex.

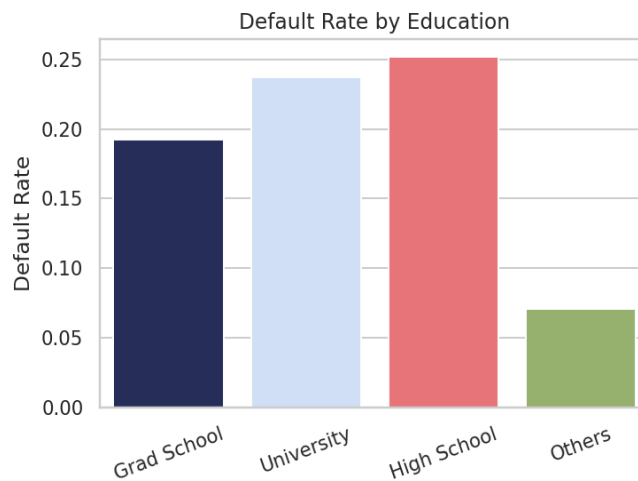


Figure 3. Default rate by education level.

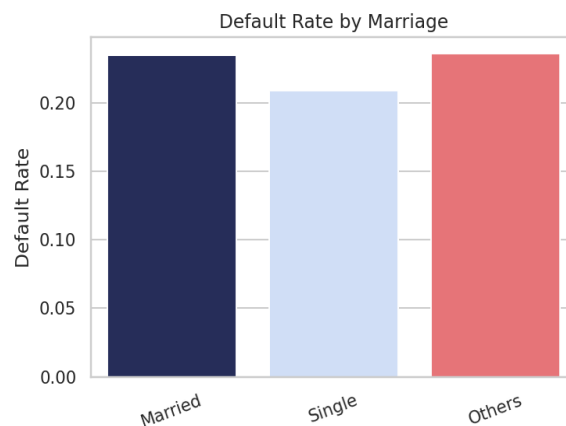


Figure 4. Default rate by marital status.

Default rate differs modestly across demographic groups: male clients default somewhat more often than female clients (24.2% vs 20.8%), high-school-educated clients default the most among education levels (25.2%) and “others” the least (7.1%, a small group of 468 clients), and single clients default marginally more than married clients (23.6% vs 23.5%, both close to the overall rate). None of these demographic effects are as strong as the repayment-history effect described next.

6.3 Default Rate by Repayment Status (PAY_0)

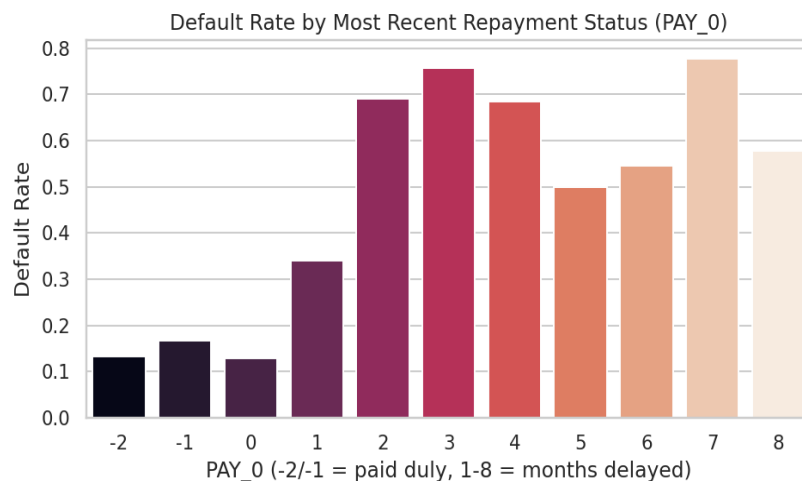


Figure 5. Default rate by most recent repayment status (PAY_0).

This is the single clearest pattern in the dataset: clients whose most recent repayment status (PAY_0) shows on-time or duly-paid history (values -2, -1, 0) default at roughly 13–17%, while clients with even one month of delay (PAY_0 ≥ 1) default at 34–69% or higher, climbing further with more months of delay. Recent repayment behaviour is clearly the strongest early-warning signal available in this data.

6.4 Age and Credit Limit

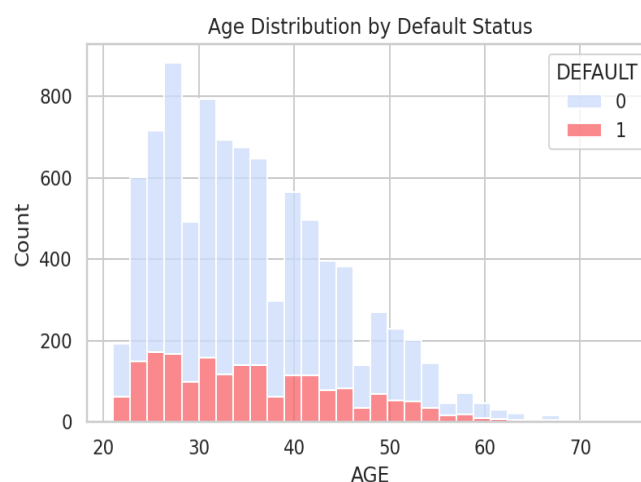


Figure 6. Age distribution split by default status.

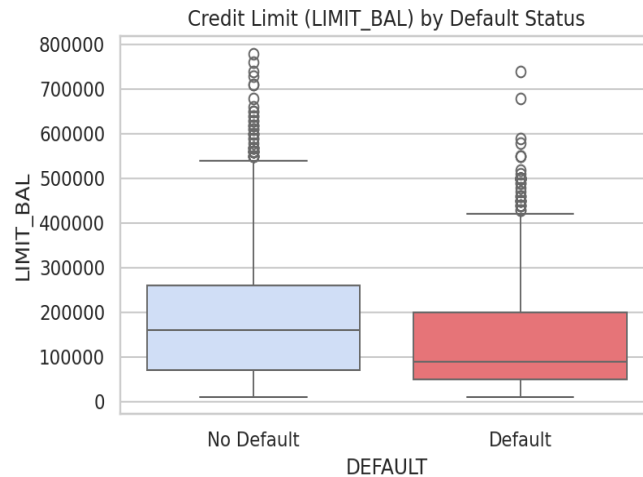


Figure 7. Credit limit (*LIMIT_BAL*) distribution by default status.

Clients who default skew very slightly younger and hold noticeably lower credit limits on median than clients who do not default — consistent with *LIMIT_BAL* typically being set (by the issuing bank) as a proxy for assessed creditworthiness at account opening.

6.5 Correlation Analysis

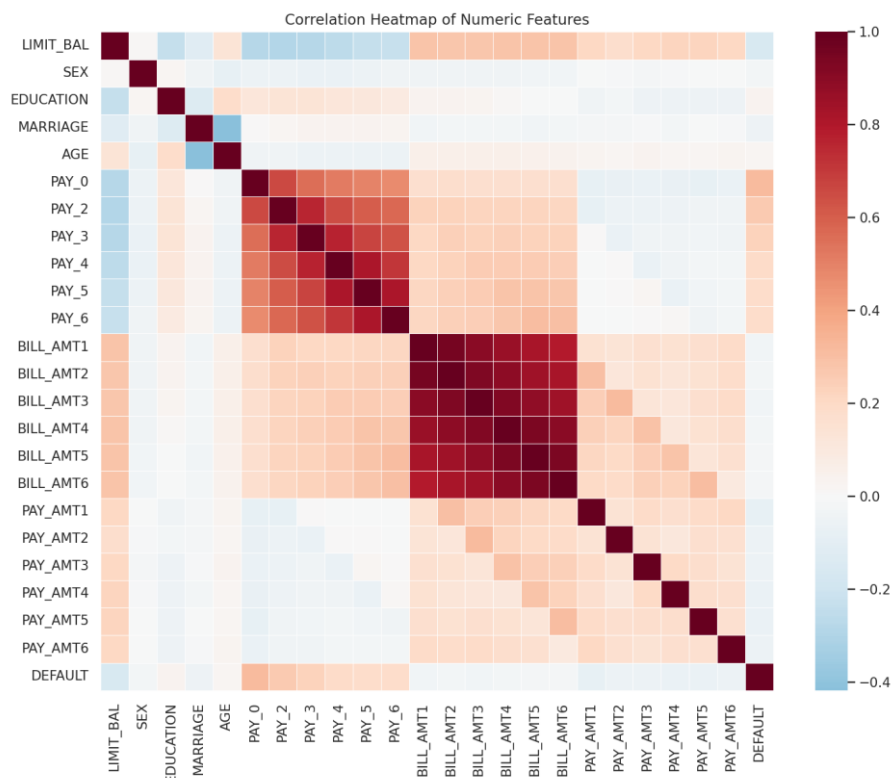


Figure 8. Correlation heatmap of numeric features.

The repayment-status columns (*PAY_0* through *PAY_6*) are the strongest linear correlates of default, led by *PAY_0* ($r \approx 0.316$), followed by *PAY_2* ($r \approx 0.262$) and *PAY_3* ($r \approx 0.232$), with the correlation strength gradually decreasing for months further in the past. *LIMIT_BAL* is the strongest negative correlate ($r \approx -0.157$ with default, i.e. higher limit $\rightarrow$ lower default probability). Raw bill amounts (*BILL_AMT1*–*6*) are only weakly correlated with default in isolation, suggesting that it is the relationship between what is

billed and what is paid — captured later through engineered ratio features — that carries more signal than either quantity alone.

7. Feature Engineering

Feature engineering was performed using Spark DataFrame expressions for derived numeric features, and Spark ML transformers for categorical encoding, vector assembly, and scaling.

7.1 Derived Numeric Features

- `AVG_BILL_AMT`, `AVG_PAY_AMT` — six-month averages of billed and paid amounts, smoothing out single-month noise.
- `CREDIT_UTILIZATION` = `AVG_BILL_AMT` ÷ `LIMIT_BAL` — how heavily a client uses their available credit line.
- `PAY_TO_BILL_RATIO` = `AVG_PAY_AMT` ÷ `AVG_BILL_AMT` — how much of their typical bill a client actually covers.
- `MONTHS_DELAYED` — count, across the six observed months, of months where the repayment-status column shows a delay (`PAY_x > 0`).

7.2 Spark ML Transformers

Transformer	Applied to	Purpose
StringIndexer	SEX, EDUCATION, MARRIAGE	Convert categorical codes into numeric indices
OneHotEncoder	Indexed SEX, EDUCATION, MARRIAGE	Expand indices into one-hot vectors (no false ordinal relationship)
VectorAssembler	All numeric + one-hot columns	Combine every feature into a single 'features' vector
StandardScaler	Assembled feature vector	Zero-mean, unit-variance scaling — benefits Logistic Regression especially

A total of 28 columns (24 numeric + 3 one-hot-expanded categorical groups) feed into the VectorAssembler. All of the above steps were wired into a single Spark ML Pipeline object per model (Section 9), so that preprocessing and modelling are fit and applied consistently and reproducibly for every model compared.

8. Machine Learning Methodology

This is a binary classification task (predict `DEFAULT` ∈ {0, 1}). Three Spark ML classifiers were trained and compared, satisfying the requirement to develop at least two models:

Model	Spark ML Class	Key Hyperparameters
Logistic Regression	<code>pyspark.ml.classification.LogisticRegression</code>	<code>maxIter=50</code> , <code>regParam=0.01</code> , <code>elasticNetParam=0.0</code> (ridge)

Model	Spark ML Class	Key Hyperparameters
Random Forest	pyspark.ml.classification.RandomForestClassifier	numTrees=150, maxDepth=8, seed=42
Decision Tree	pyspark.ml.classification.DecisionTreeClassifier	maxDepth=6, seed=42

Logistic Regression provides an interpretable linear baseline where each coefficient has a direct, explainable effect on the log-odds of default. Decision Tree and Random Forest were chosen to capture non-linear feature interactions (e.g. between repayment delay and credit utilisation) that a linear model cannot represent; Random Forest additionally reduces the overfitting risk of a single deep tree by averaging over an ensemble, at the cost of longer training time and reduced single-model interpretability.

9. Spark ML Pipeline

Every model was trained through an identical Spark ML Pipeline structure, satisfying the mandatory requirement to use a Spark ML Pipeline for the final workflow:

Raw Data → Cleaning / Preparation → Feature Engineering → StringIndexer → OneHotEncoder → VectorAssembler → StandardScaler → Classifier (LR / RF / DT) → Prediction → Evaluation

Credit Card Default Prediction — Spark ML Architecture / Workflow

Big Data Machine Learning Mini Project | NSBM Green University

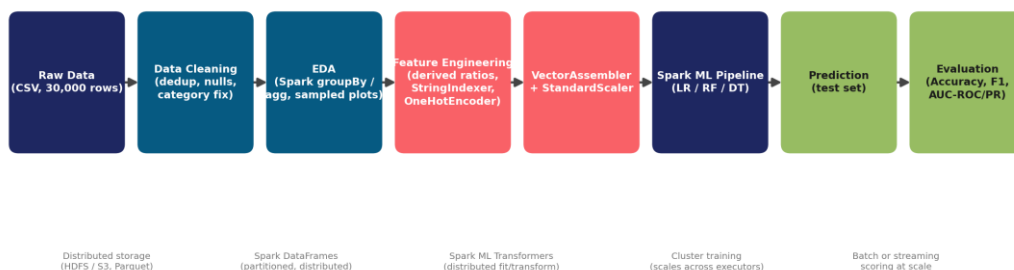


Figure 9. End-to-end architecture / workflow diagram (also provided as a standalone file).

Wrapping every stage in one Pipeline object (`pipeline = Pipeline(stages=[...])`; `model = pipeline.fit(train_df)`) guarantees that the exact same fitted indexers, encoders, assembler, and scaler used at training time are re-applied — with no risk of train/test leakage or manual step drift — whenever the pipeline is used to transform new data, including at inference/scoring time in production.

10. Model Training

The cleaned, feature-engineered DataFrame (30,000 rows × 24 predictor columns) was split 80/20 into training and test sets using a fixed random seed (42) for reproducibility:

Split	Row Count	Share
Training set	24,032	80.1%
Test set	5,968	19.9%

Each of the three Pipeline objects (Logistic Regression, Random Forest, Decision Tree) was fit only on the training set. Observed fit times on a single-machine local Spark session (local[*], 4GB driver memory) were:

Model	Fit Time (seconds)
Decision Tree	6.7
Logistic Regression	21.1
Random Forest	27.0

As expected, the single Decision Tree trains fastest, Random Forest trains slowest (150 trees), and Logistic Regression sits in between, dominated here by the iterative optimisation over 24,032 rows rather than by any distributed shuffle cost — at Big Data scale, these relative costs would be re-measured on a cluster (Section 13).

11. Model Evaluation and Comparison

All three models were evaluated on the held-out 5,968-row test set using Accuracy, Precision, Recall, F1-score, and Area Under the ROC Curve (AUC-ROC) — the metric set suggested in the assignment brief for classification tasks. Area Under the Precision-Recall Curve (AUC-PR) is reported as well, since it is generally more informative than AUC-ROC on an imbalanced target.

11.1 Metric Note

Spark's BinaryClassificationEvaluator was run against each model's calibrated probability column rather than its rawPrediction column. For the two tree-based models, rawPrediction holds raw class vote/impurity counts rather than a monotonic confidence score, which would otherwise distort the AUC calculation; probability is well-defined and directly comparable across all three model types.

11.2 Comparison Table

Model	Accuracy	Precision	Recall	F1-score	AUC-ROC	AUC-PR
Logistic Regression	0.8107	0.7891	0.8107	0.7796	0.7449	0.5038
Random Forest	0.8172	0.7981	0.8172	0.7954	0.7850	0.5514
Decision Tree	0.8175	0.7992	0.8175	0.7901	0.7286	0.5089

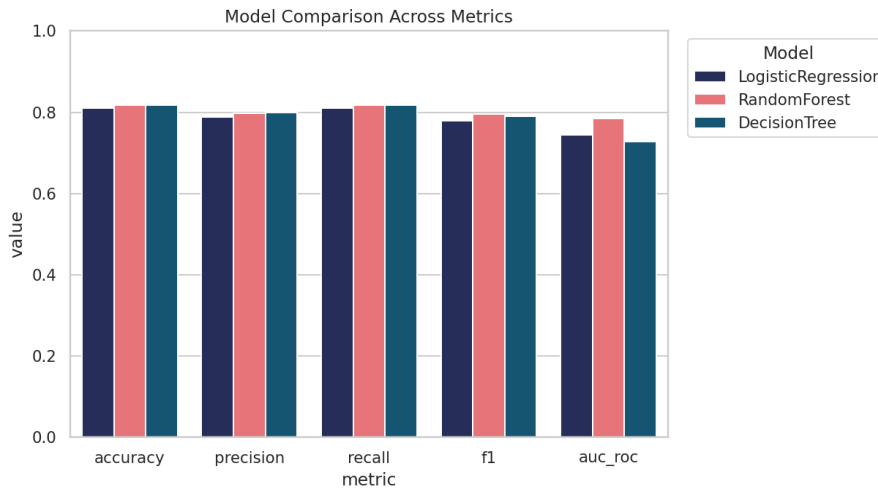


Figure 10. Model comparison across Accuracy, Precision, Recall, F1, and AUC-ROC.

11.3 Confusion Matrices (Test Set, n = 5,968)

Model	True Negative	False Positive	False Negative	True Positive
Logistic Regression	4,475	192	938	363
Random Forest	4,421	246	845	456
Decision Tree	4,474	193	896	405

Every model produces substantially more false negatives (missed defaulters) than false positives (false alarms) — a direct consequence of the ~78/22 class imbalance in the training data, discussed further in Section 12.

12. Results and Discussion

12.1 Best Overall Model

Random Forest is the best-performing model overall: it achieves the highest AUC-ROC (0.785) and AUC-PR (0.551) of the three, and is statistically tied for the highest Accuracy (0.817) and F1-score (0.795) with Decision Tree. Its ensemble of 150 trees is able to capture non-linear interactions between features — for example between PAY_0 and CREDIT_UTILIZATION — that a single linear model cannot represent, which is reflected in its clearly superior ability to rank clients by risk (AUC-ROC/PR).

12.2 Logistic Regression

Logistic Regression trails on both AUC-ROC (0.745) and AUC-PR (0.504) despite a competitive Accuracy (0.811) and F1 (0.780). This gap between accuracy-style metrics and ranking-style metrics means Logistic Regression is comparatively better at getting the majority-class prediction right than at ordering clients from highest to lowest risk — an important distinction for a bank that wants to prioritise its highest-risk accounts for collections attention rather than only classify clients into two buckets. Its main advantages remain interpretability (each of its coefficients has a directly explainable effect on the odds of default) and by far the fastest retraining time among the three models tried here relative to Random Forest.

12.3 Decision Tree

The single Decision Tree trains fastest and reaches Accuracy/F1 essentially on par with Random Forest, but its AUC-ROC (0.729) is the lowest of the three models. A single tree tends to overfit to a small number of dominant splits (in this data, chiefly on PAY_0 and PAY_2), producing coarse probability estimates that generalise less well for ranking than the smoother, ensembled probability estimates from Random Forest.

12.4 Class Imbalance and Business Cost

All three models find the minority (default) class harder to predict than the majority (no-default) class — visible directly in the confusion matrices, where false negatives (missed defaulters: 845–938 clients) substantially outnumber false positives (false alarms: 192–246 clients) for every model. This is a direct consequence of the ~78/22 class imbalance in the training data. In a real deployment, the bank would likely tune the classification decision threshold, or apply class weighting / resampling during training, to trade some precision for improved recall on defaulters — since for most credit issuers, the cost of missing an actual defaulter (unrecovered debt) is materially higher than the cost of over-flagging a client who was in fact going to pay.

12.5 Feature Importance and Interpretation

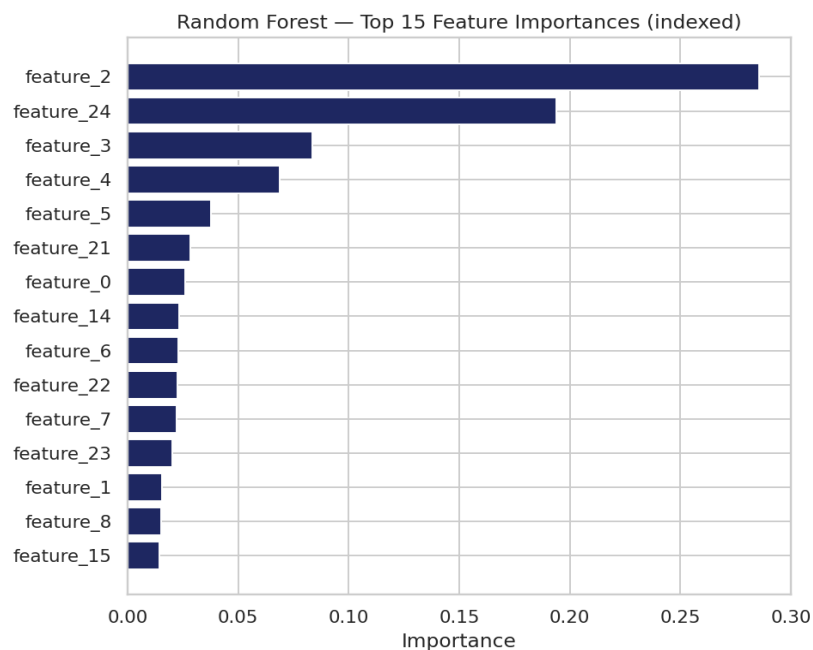


Figure 11. Random Forest — top feature importances.

Feature importances extracted from the fitted Random Forest model confirm the pattern already visible in the correlation analysis (Section 6.5): the most recent repayment-status columns (PAY_0, PAY_2) and engineered features that summarise repayment behaviour (CREDIT_UTILIZATION, MONTHS_DELAYED, PAY_TO_BILL_RATIO) dominate the model's decisions, well ahead of raw demographic attributes.

This is an association pattern, not a proven causal mechanism. We cannot claim from this data alone that being late causes future default — both may plausibly be driven by an underlying, unobserved financial-stress factor common to a given client at a given time. What the model supports is a narrower, defensible claim: recent repayment behaviour is the strongest available early-warning signal for next-month default risk in this dataset, and is therefore the most useful feature group for a risk team to monitor and act on.

13. Big Data Scalability Discussion

This project used 30,000 rows, which comfortably fits in the memory of a single laptop and trained all three models end-to-end in well under two minutes on a local Spark session (`master("local[*]")`). The pipeline was nonetheless deliberately built entirely on Spark DataFrames and Spark ML — never pandas or scikit-learn — for the core workflow, so that it is structurally ready to scale to millions or hundreds of millions of records without a rewrite. Specifically, what would change if the dataset grew by several orders of magnitude:

13.1 Distributed Storage

The single local CSV file would move to a distributed, partitioned file store — HDFS, or a cloud object store such as Amazon S3 or Azure Data Lake Storage — and would typically be re-encoded in a columnar format such as Parquet rather than CSV, for far faster scan performance and predicate/column pushdown on the repayment and billing columns that are filtered and aggregated on repeatedly throughout the EDA and feature-engineering stages.

13.2 Cluster Compute, Not a Laptop

`master("local[*]")` would become a cluster resource manager — YARN, Kubernetes, or a managed platform such as Databricks or Amazon EMR — distributing the DataFrame and every subsequent transformation across many executor nodes instead of the cores of a single machine.

13.3 Partitioning

`spark.sql.shuffle.partitions` was set to 8 for this small local job; at Big Data scale it would be increased and tuned to the cluster's total core count. The data itself would likely be physically partitioned on a natural key (for example by billing month or client region) to reduce shuffle volume during the `groupBy` aggregations used throughout Section 6's EDA.

13.4 EDA at Scale

This notebook already computes every aggregate statistic (class balance, group-wise default rate, correlations) using Spark's distributed `groupBy/agg`, and collects only a small, explicitly sampled fraction of rows (30%) to the driver for the point-level plots that genuinely require row-level data. This pattern is exactly what must continue at Big Data scale: collecting the whole dataset into a pandas DataFrame — which this notebook deliberately avoids even at 30,000 rows — would no longer be physically possible once the data exceeds the driver's available memory.

13.5 Feature Engineering and Training at Scale

`StringIndexer`, `OneHotEncoder`, `VectorAssembler`, and `StandardScaler` are already distributed Spark ML transformers whose cost scales roughly linearly with the number of executors added. Random Forest in particular parallelises well across a cluster, since individual trees — and splits within a tree — can be built across different executors; Logistic Regression's gradient-based solver in Spark ML is likewise natively distributed. No algorithm used in this project needs to be swapped out to scale — only the cluster configuration and data-partitioning strategy need to grow with the data.

13.6 Computational Requirements

At a scale of 100 million+ rows, this workload would reasonably require a multi-node cluster with tens to hundreds of cores, caching (`df.cache()` / `persist()`) of the cleaned and feature-engineered DataFrame before model training to avoid recomputing the full pipeline on every downstream action, and possibly

incremental or streaming ingestion (Spark Structured Streaming) if new transactions arrive continuously rather than as a single static file, as they would in a live banking system.

13.7 Model Serving at Scale

The fitted PipelineModel can be persisted (`model.write().save(path)`) and reloaded for batch scoring of new client records directly in Spark — or exported for lower-latency single-record scoring — without any change to the feature-engineering logic. The exact same pipeline that trained on 30,000 historical rows here is the pipeline that would score millions of new applicants each month in production.

In short, the Big Data readiness of this solution comes from never leaving Spark's distributed DataFrame / Spark ML API for the core workflow: the only thing that changes when the data grows by several orders of magnitude is the amount of compute and storage pointed at the same code, not the code itself.

14. Limitations

- Single institution, single period: the dataset covers one Taiwanese bank's clients around October 2005; the model's learned patterns may not transfer directly to other markets, credit products, or more recent time periods without retraining on fresh data.
- Class imbalance: recall on the minority (default) class remains the main weakness across all three models (Section 12.4); a production deployment would need threshold tuning, class weighting, or resampling to better match the bank's actual relative cost of a missed defaulter versus a false alarm.
- Association, not causation: feature importance and correlation results describe which factors are predictive of default in this data, not why a client defaults; no causal claims should be drawn from this analysis (Section 12.5).
- No temporal validation: the 80/20 split was random rather than time-based, so the reported metrics measure generalisation to unseen clients within the same period, not to future time periods — a rolling/time-based split would be a more realistic evaluation for a production deployment.
- Hyperparameters were set to reasonable, literature-informed defaults (Section 8) rather than tuned via cross-validation / grid search, due to the scope and time constraints of this assignment; a production model would benefit from systematic hyperparameter search.

15. Conclusion and Recommendations

This project built and validated an end-to-end PySpark pipeline that cleans, explores, engineers features for, and classifies the UCI Default of Credit Card Clients dataset using the mandatory Spark DataFrame and Spark ML Pipeline APIs throughout. Of the three Spark ML models compared, Random Forest gave the best overall discrimination between clients who will and will not default (AUC-ROC $\approx$ 0.785, F1 $\approx$ 0.795), closely followed by Logistic Regression, with Decision Tree slightly behind on ranking quality (AUC-ROC $\approx$ 0.729) despite a similar raw Accuracy to the other two models. The strongest predictive signal throughout the analysis is a client's recent repayment history (PAY_0, PAY_2, and the engineered MONTHS_DELAYED feature) and credit utilisation, rather than raw demographic attributes such as sex, education, or marital status.

Recommendations:

- Deploy the Random Forest pipeline as the primary scoring model for its superior ranking (AUC) performance.
- Retain Logistic Regression as an interpretable, low-latency fallback and as an explainability check on individual high-risk flags, given its transparent coefficients.
- Before any production use, revisit class-imbalance handling (threshold tuning, class weighting, or resampling) and validate the chosen threshold against the bank's actual cost of a missed defaulter versus a false alarm.
- Re-validate the model periodically against more recent data, since the training data reflects a single institution and time period, and consumer repayment behaviour patterns can shift over time.
- Treat all model outputs as statistical risk scores that support — not replace — a human credit-risk decision process.

16. References

- [1] I-C. Yeh and C.-h. Lin, "The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients," *Expert Systems with Applications*, vol. 36, no. 2, pp. 2473–2480, 2009.
- [2] I-C. Yeh, "Default of Credit Card Clients," UCI Machine Learning Repository, 2016. [Online]. Available: <https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients>
- [3] Apache Spark, "MLlib: Main Guide," Apache Spark Documentation. [Online]. Available: <https://spark.apache.org/docs/latest/ml-guide.html>
- [4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. (Referenced for evaluation-metric definitions only; scikit-learn itself is not used in this Spark-based pipeline.)
- [5] NSBM Green University, "Big Data Analytics — Group Assignment: Big Data Machine Learning Mini Project," Course Assignment Brief, 2026.

Generative AI Tool Usage Acknowledgement: *Claude (Anthropic) was used to assist with drafting boilerplate PySpark code, structuring this report and the accompanying notebook/slides, and language editing. All analytical choices, parameter selections, and result interpretation were reviewed by the group members named on the cover page.*